

Optimización y paralelización de cómputo para extracción de características de estrellas variables periódicas

Autor

- Felipe Clariá Dambolena

Directores

- Dr. Juan Bautista Cabral
- Dr. Bruno Orlando Sánchez

FAMAF, Universidad Nacional de Córdoba

Mayo de 2025



Contenidos

1. Introducción
 2. feets
 3. Motivación de este trabajo
 4. Modernización de feets
 - a. Paralelización
 - b. Reingeniería
 - c. Integración con LightCurve
 5. Resultados
 6. Conclusiones
-

1. Introducción

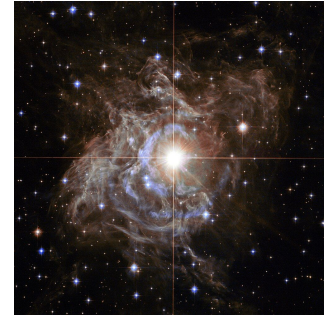
Estrellas variables periódicas

Estrellas cuyo brillo varía a lo largo del tiempo, con aumentos y disminuciones periódicos

Actúan como **candelas estándar**.

Usadas para:

- Estudiar la evolución estelar
- Determinar la estructura de la Vía Láctea
- Estimar parámetros cosmológicos



Cefeida (RS Puppis). Créditos:

<https://esahubble.org/images/heic1323a>

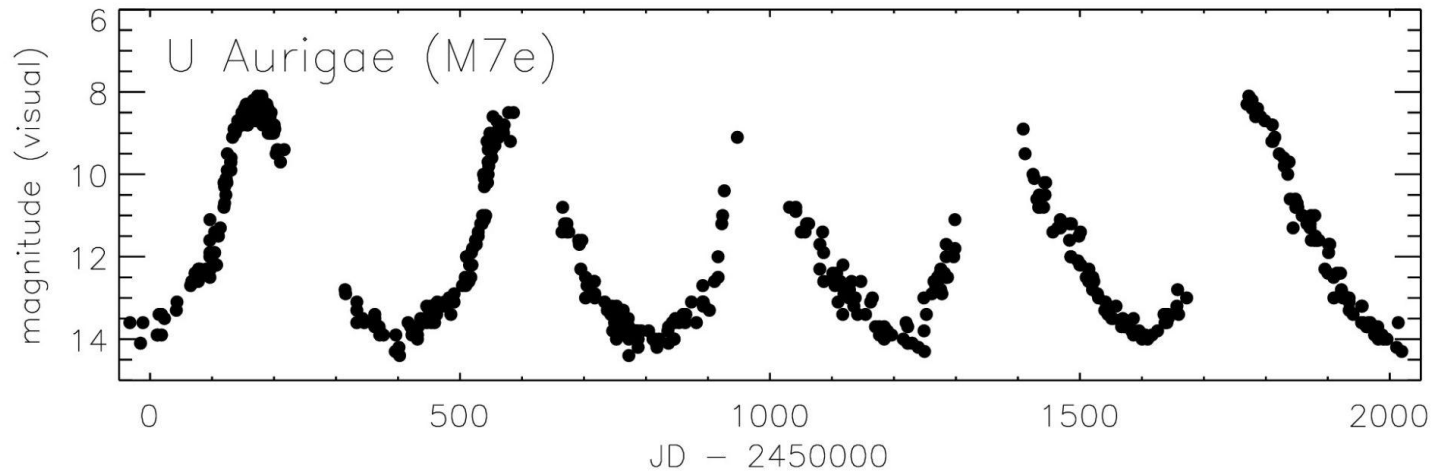


RR Lyrae. Créditos:

<https://aladin.cds.unistra.fr/aladin.gml>

Curvas de luz

Serie temporal del brillo de una estrella obtenida a partir de observaciones fotométricas repetidas.



Curva de luz de U Aur, variable Mira de tipo M. Créditos: Templeton et al. (2005)

Astronomía observacional en la actualidad

Vivimos en la era del **"Big Data"**.

Relevamientos astronómicos masivos:

- VVV, OGLE, Gaia, LSST (Vera Rubin), entre otros.
- El VVV (*Vista Variables in the Via Lactea*) ha producido hasta más de 300 GB por noche, con millones de curvas de luz a lo largo de los años.

Problema: analizar manualmente estos datos se hace inviable.



Aprendizaje automático (ML)

“Se dice que un programa de computadora aprende de la experiencia E con respecto a una clase de tareas T y una medida de desempeño P, si su desempeño en las tareas T, medido por P, mejora con la experiencia E.”

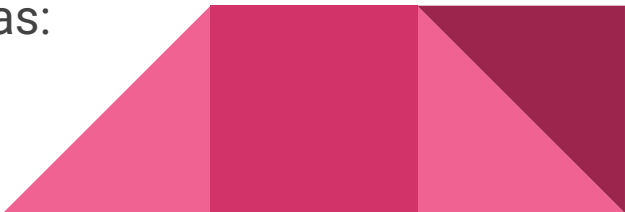
– Tom Mitchell (1997)

En el análisis de curvas de luz:

- Clasificar estrellas variables
- Estimar propiedades físicas
- Detectar comportamientos atípicos

Los modelos de ML no trabajan directamente sobre curvas:

- **Características**



Extracción de características

Las características capturan aspectos clave de la variabilidad.

La calidad del modelo depende del proceso de selección de características.

Ejemplos:

- Período, amplitud, asimetría, y muchas otras.

Existen herramientas para facilitar el proceso:

- FATS, **feets**, LightCurve





2. feets

FEature Extractor for Time Series (feets)

Biblioteca de Python destinada a la **extracción de características** de series temporales, y en especial para las **curvas de luz**.

Colección de **extractores**, cada uno produce una o más características específicas.

El usuario selecciona las **características** deseadas y proporciona los datos de la curva como parámetros de entrada.



Funcionalidades

- Más de 40 características estadísticas y astronómicas listas para calcular:

Ejemplos: media, desviación estándar, período, amplitud, ...

- *Framework* especializado para desarrollar y registrar nuevos extractores de características.
- Limpieza y preprocesamiento de datos.
- Lector de catálogos astronómicos: OGLE-III, MACHO.



Extractores

Unidades que calculan una o más características a partir de una curva de luz y/o de características obtenidas por otros extractores.

Cada extractor debe especificar al menos tres cosas:

- Las **características** que calcula
- Los **datos de la curva de luz** sobre los que trabaja
- Un **método de extracción**



Extractores

Extractor	Datos	Características
Mean	time, magnitude	Mean
Std	time, magnitude	Std
Amplitude	magnitude	Amplitude
LombScargle	time, magnitude	PeriodLS, Period_fit, Psi_CS, Psi_eta

Limitaciones

Ejecución secuencial:

- Los extractores se ejecutan de a uno a la vez
- Cuello de botella para volúmenes de datos muy grandes.

Usabilidad limitada: Los extractores trabajan sobre una única curva de luz.

Stack tecnológico obsoleto:

- Python 2.
- Dependencias desactualizadas.



Motivación de este trabajo

Modernizar *feets* para abordar las limitaciones encontradas.

Objetivos:

1. Optimizar el rendimiento en contextos de análisis astronómico masivo
2. Simplificar el código y mejorar su estructura interna para mejorar la calidad del *software*.



3. Modernización de *feets*

3.a. Paralelización

Paralelización

- Dividir el problema en subproblemas más chicos, que puedan ser resueltos de forma independiente y en simultáneo.
- Maximizar potencialmente el uso de los recursos del sistema y acortar los tiempos de ejecución.

Idea: Ejecutar todos los extractores a la vez, en paralelo.

Problema: Extractores con dependencias



Extractores con dependencias

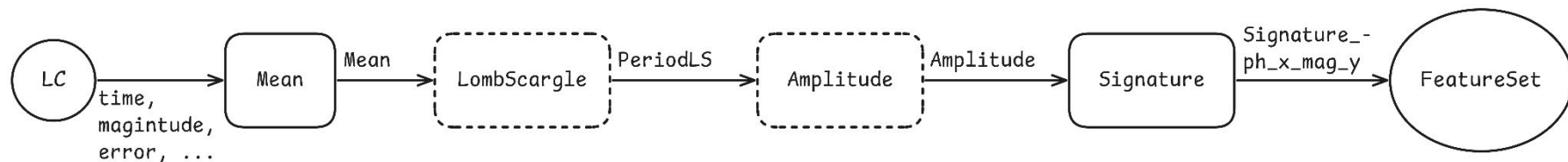
Algunos extractores dependen de las características computadas por otros.

Ejemplo: El extractor **Signature** depende de **PeriodLS** y **Amplitude**.

Todas las dependencias de un extractor deben ser calculadas antes de comenzar su ejecución.



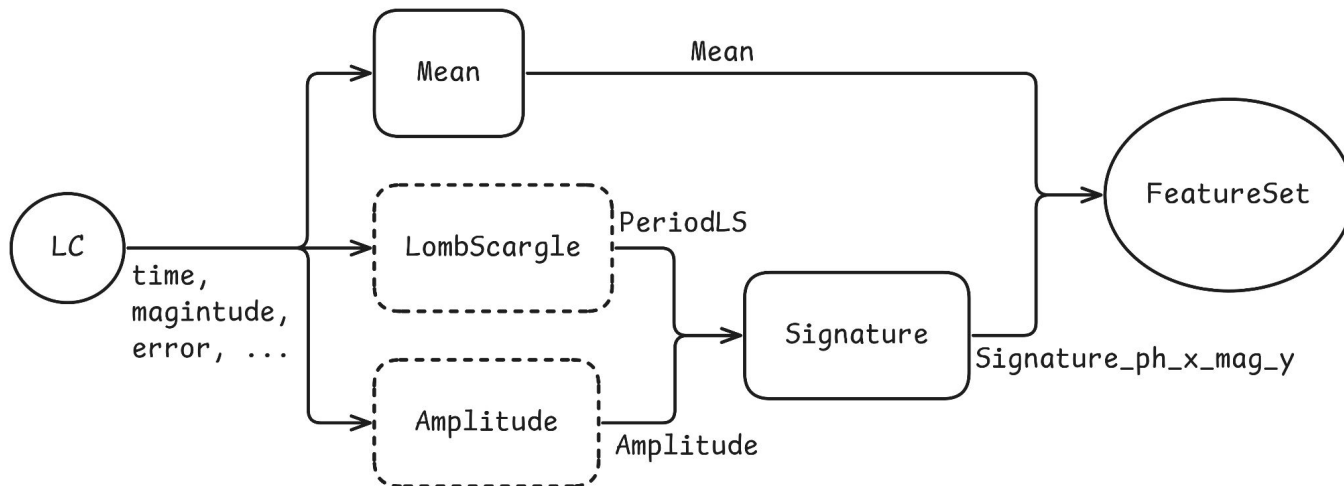
La versión secuencial



Cadena de extracción secuencial.

Se ejecutan los extractores uno tras otro, siguiendo el orden de sus dependencias.

La nueva versión



Grafo de extracción.

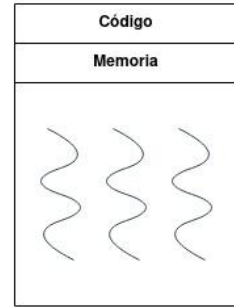
Los extractores independientes entre sí se ejecutan a la vez.

Los extractores que dependen de otros esperan a la finalización de sus dependencias.

Estrategias de ejecución

Multithreading:

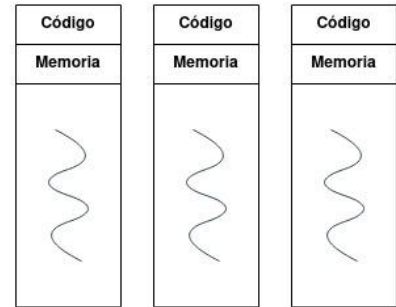
- Varios hilos dentro de un mismo proceso
- En python está limitado por el Global Interpreter Lock (GIL)



Multithreading

Multiprocessing:

- Varios procesos independientes.
- Cada proceso tiene su propio intérprete, evitando el GIL.



Multiprocessing

Dask

Biblioteca para la paralelización y distribución de tareas.



Se integra fácilmente en el ecosistema de Python.

Construcción y ejecución eficiente de grafos de tareas paralelas con dependencias.

Schedulers:

- *Single-machine*: Con estrategias de **multithreading** y de **multiprocessing**.
- *Distribuido*: Permite escalar a múltiples máquinas.

3.b. Reingeniería

Rediseño de extractores

Reimplementación de la clase base **Extractor**:

- Especificación simplificada con menos *boilerplate*.
- Lógica interna para la integración con el modelo paralelo y otras funcionalidades.



Nuevas funcionalidades

Soporte para operar sobre **múltiples curvas de luz a la vez**.

Guardar/cargar configuraciones del espacio de trabajo.

Nueva interfaz para el **análisis de resultados**.

... y más.



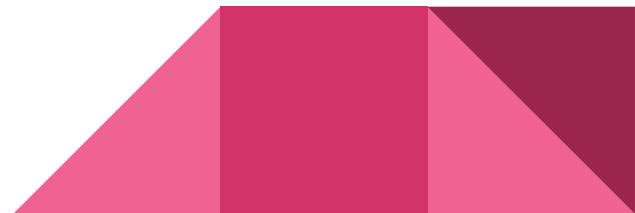
Otras mejoras

Separación en módulos.

Actualización del stack tecnológico:

- Python 3.13
- AstroPy v7.0
- NumPy v2.2.0

Estándar de código PEP-8



3.c. Integración con LightCurve

LightCurve

Biblioteca moderna para la extracción de características de curvas de luz.

- Implementada en **Rust**
- También tiene una versión alternativa en Python.

Ambas versiones ofrecen tiempos de ejecución más rápidos que feets.

Cuenta con características ausentes en feets:

- **BazinFit, ReducedChi2, WeightedMean, ...**



Integración

Motivación: Combinar la velocidad de LightCurve con la flexibilidad de feets.

Incorporamos todos los extractores de LightCurve en feets, y en caso de tener características repetidas dejamos la versión optimizada de LightCurve.

Incorporación de un nuevo tipo de datos: **flujo**.



4. Resultados

4.a. Tiempos de ejecución

Análisis

Medimos los tiempos de extracción sobre grupos de una o más curvas de luz simuladas, variando diferentes parámetros:

- **Longitud de las curvas de luz:** 30, 100, 300 y 1000 observaciones por curva.
- **Tamaño de los grupos:** 1, 50, 100, 500 y 1000 curvas por grupo.
- **Scheduler de Dask:**
 1. **synchronous** (sin paralelismo),
 2. **threads** (*multithreading*)
 3. **processes** (*multiprocessing*)



Análisis

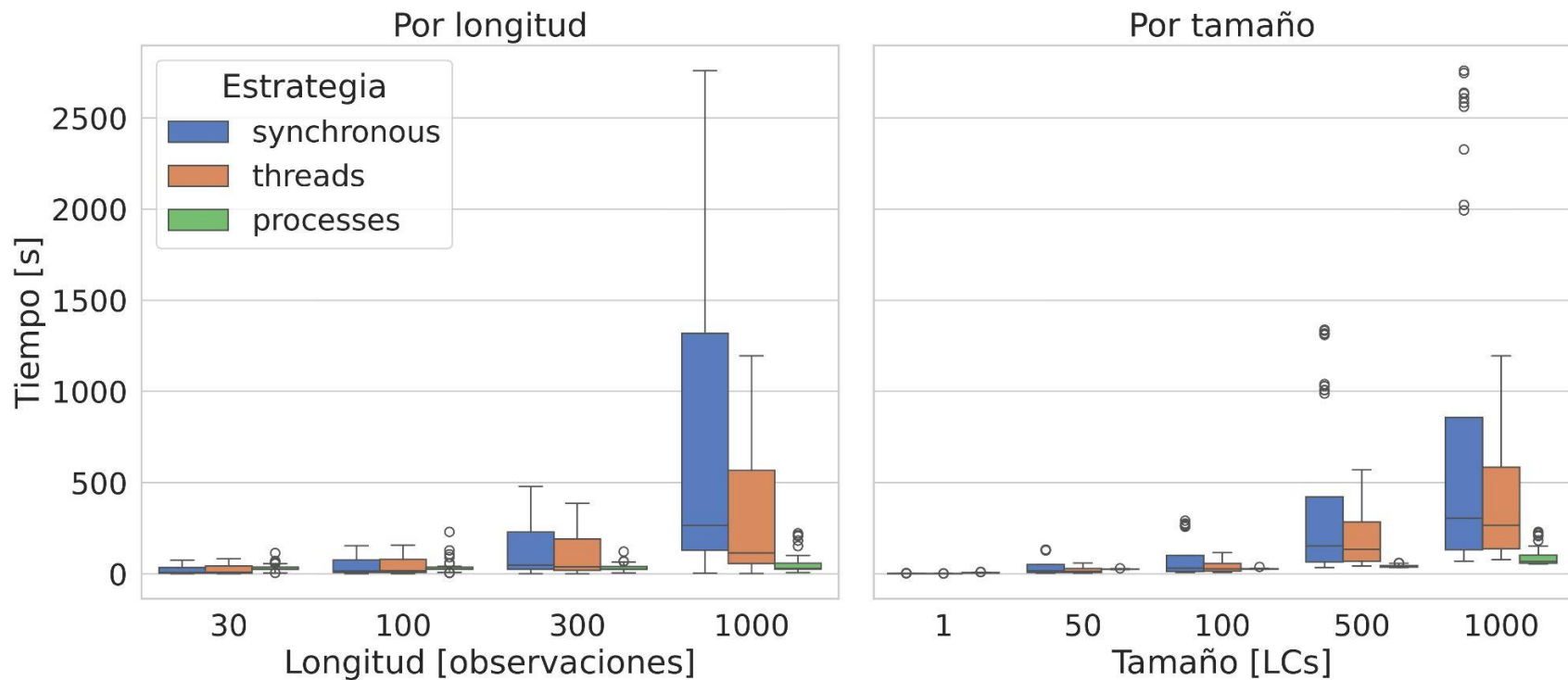
Repetimos el experimento con las mismas curvas sobre dos selecciones de características distintas:

- **Selección completa:** todas las características disponibles.
- **Selección reducida:** sin las nuevas características de LightCurve.

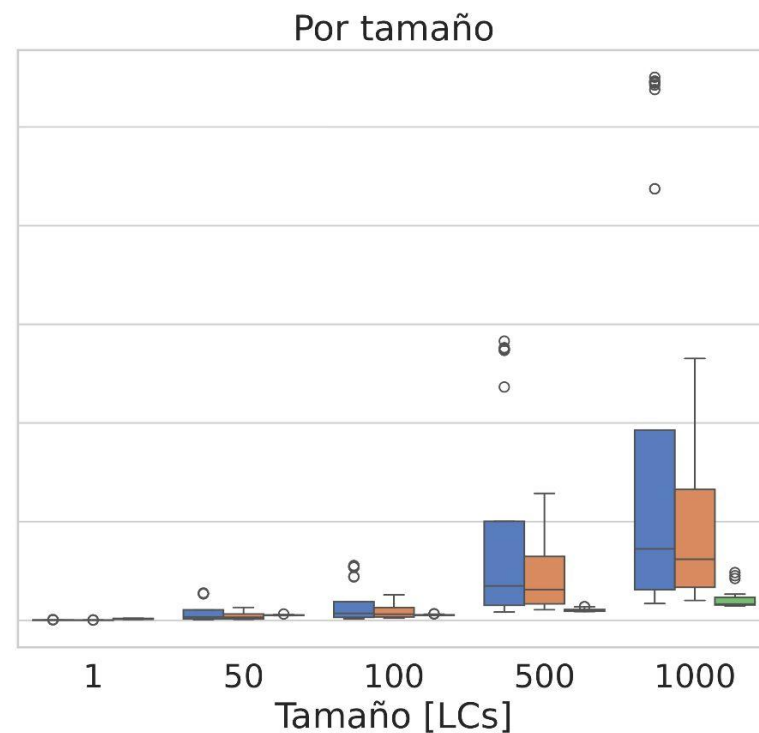
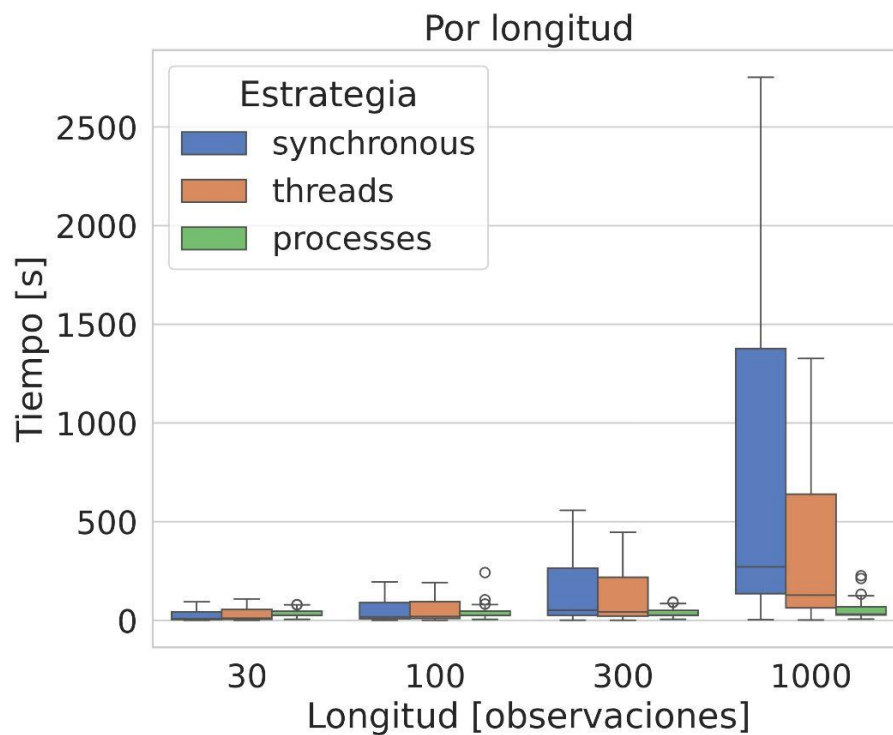


El experimento se llevó a cabo en la supercomputadora Serafín de la UNC.

Tiempos de extracción (selección completa)

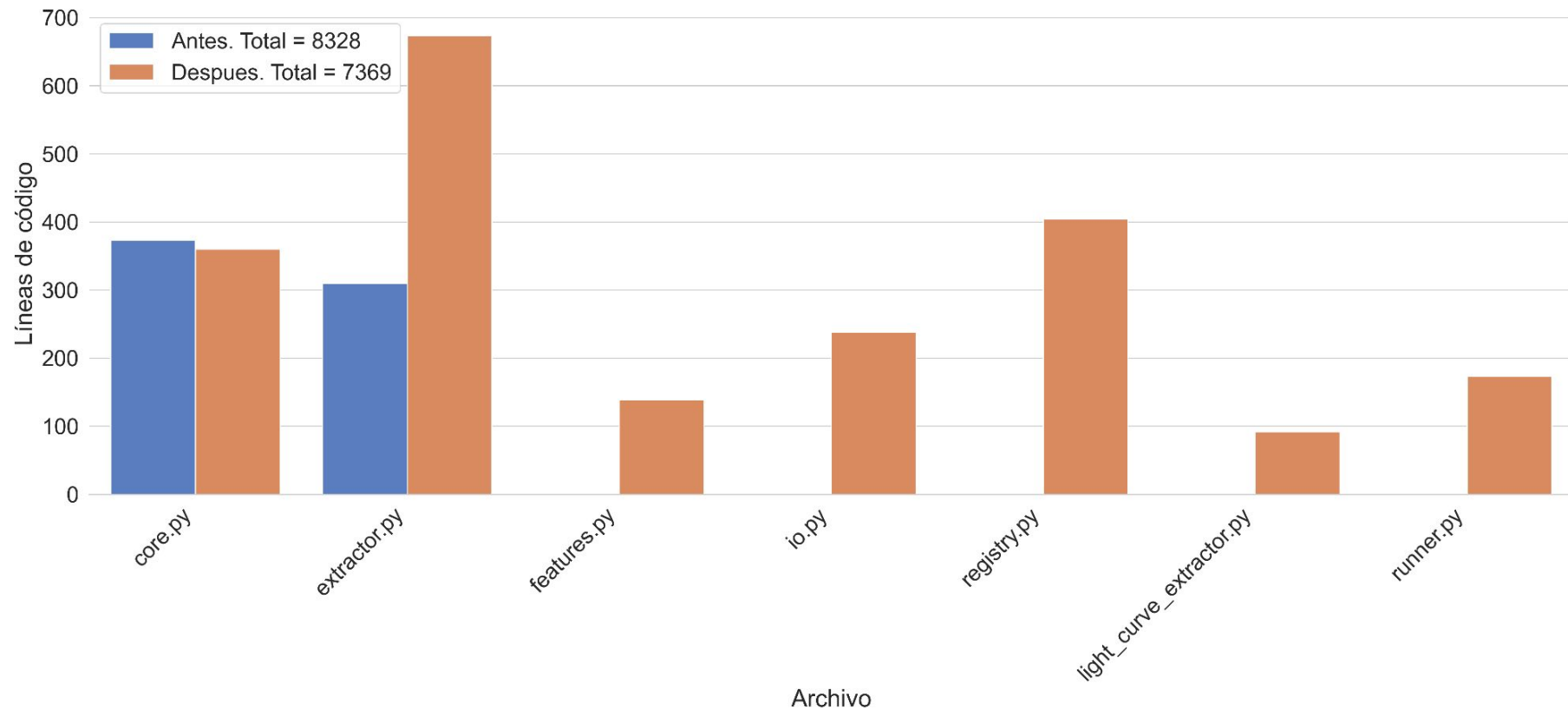


Tiempos de extracción (selección reducida)

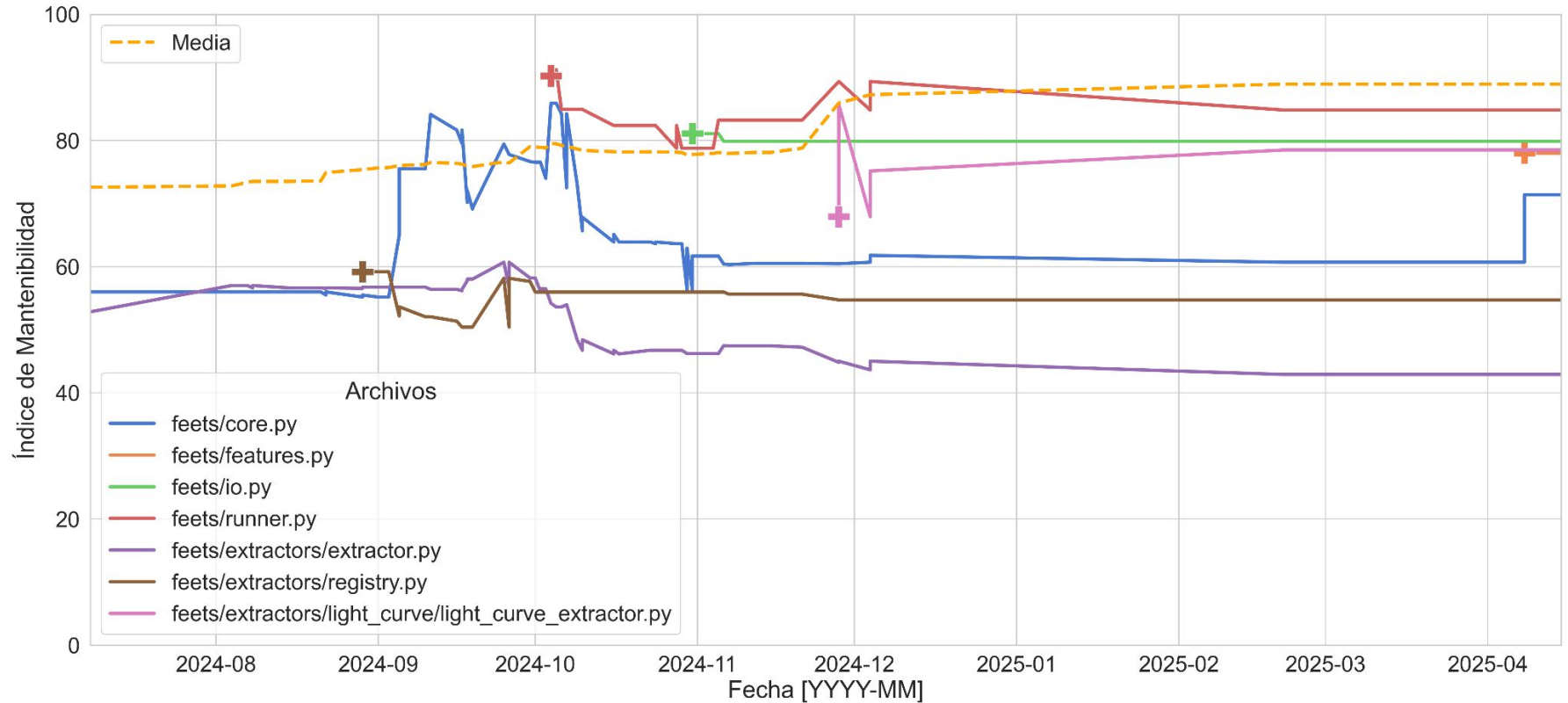


4.b. Análisis estático

Líneas de código históricas en feets



Índice de mantenibilidad histórico de feets



5. Conclusiones

Conclusiones

- Reducción significativa de tiempos de ejecución.
- Mejora general de calidad y mantenibilidad.
- Nuevas funcionalidades que la hacen más flexible y robusta.

Como resultado, la nueva versión de feets es una herramienta **moderna, eficiente y preparada** para enfrentar los desafíos del análisis de series temporales en la era del Big Data, **sin comprometer la calidad del software**.



Trabajo futuro

Probar el desempeño con el scheduler `distributed` de Dask sobre una red distribuida.

Extender el análisis de características con la capacidad de visualizar los resultados obtenidos mediante gráficos.





Muchas gracias

¿Preguntas?

Métricas

- Complejidad ciclomática (**CC**)
- Líneas de código (**LOC**)
- Porcentaje de comentarios (**POC**)
- Volumen de Halstead (**V**)
- Índice de mantenibilidad (**MI**): Involucra todas las anteriores

$$MI_{\text{radon}} = \max \left[0, \frac{100}{171} (171 - 5,2 \ln V - 0,32CC - 16,2 \ln LOC + 50 \sin \sqrt{2,4POC}) \right]$$

